	Critères	MongoDB	Cassandra	Redis	Neo4j	DynamoDB
	Flexibilité du schéma	Modèle orienté document (BSON) permettant d'imbriquer des données complexes sans schéma prédéfini.	Structure en colonnes larges imposant une définition stricte des clés de partitionnement dès le départ.	Très flexible car simple (Clé-Valeur), mais limite le stockage de structures de données très riches.	Idéal pour ajouter des types de relations ou de nœuds sans impacter les données existantes.	Schéma souple, mais la modification des index globaux après la création peut être complexe.
	Vitesse de dév. (2 mois)	La manipulation d'objets JSON correspond nativement aux langages modernes (JS, Python). Itérations ultra-rapides.	Demande de modéliser les tables en fonction des requêtes finales (Query-first design), ce qui punit l'erreur.	API extrêmement simple. Très peu de "code de plomberie" à écrire pour stocker/récupérer.	Nécessite de maîtriser le langage Cypher, très puissant mais différent du SQL ou du JSON classique.	Intégration directe via SDK, mais la gestion des permissions IAM et des limites peut ralentir le début.
	Facilité d'Ops (Setup)	MongoDB Atlas propose un cluster gratuit en 3 clics avec monitoring et sauvegardes automatisés.	Demande une configuration lourde (JVM, RAM) et une gestion complexe du cluster, même en local.	S'installe en une ligne de commande ou via un service managé très léger.	Nécessite une configuration mémoire précise pour éviter les ralentissements sur les gros graphes.	Zéro serveur à gérer (Serverless). AWS s'occupe de tout, pas d'installation nécessaire.
	Puissance de requêtage	Supporte les jointures ($lookup), les agrégations poussées, la géolocalisation et la recherche textuelle.	Pas de jointures, pas de fonctions d'agrégation nativement simples. On ne peut requêter que par clé.	Se limite à des opérations simples sur des clés. Pas de recherche complexe sur le contenu des valeurs.	Capacité unique à traverser des millions de relations (liens) en un temps record.	Efficace sur les clés, mais les scans (recherche hors index) deviennent vite très coûteux et lents.
	Courbe d'apprentissage	Très proche de la logique de programmation objet. Documentation massive et accessible.	Concepts de systèmes distribués (Théorème CAP, Gossip) indispensables pour ne pas casser la base.	Se comprend en moins d'une heure. Très peu de concepts théoriques à assimiler.	Demande un changement de paradigme pour "penser en graphe" plutôt qu'en listes ou tables.	Bien que simple au début, le "Single Table Design" demande une expertise pour être efficace.
		100,00%	**28,00%**	**72,00%**	**76,00%**	**76,00%**